



UNIVERSIDAD
DE ALMERÍA

PROCESOS DE INGENIERÍA DEL SOFTWARE 2

Securización de Aplicaciones

GRUPO 7

Andrés Ruiz Andújar

Sergio Gómez Vico

Adrián Martínez Granados

Raúl Martínez Gutiérrez

Índice de Contenidos

1. Introducción	4
2. ISO/IEC 27034	4
2.1. ¿Por qué es importante la ISO 27034?	4
2.2. Objetivo de ISO/IEC 27034	5
2.3. Cuadro normativo de la organización interesada en ISO/IEC 27034.....	5
2.4. Estructura del estándar: sus partes.....	6
3. El ONF (Organizational Normative Framework).....	6
3.1 Definición	6
3.2 Componentes del ONF	7
3.4 Por qué el ONF es central en la seguridad de aplicaciones	7
3.5 El ONF en el ciclo de vida de la aplicación	8
4. Ciclo de Vida de Desarrollo Seguro (S-SDLC)	8
4.1. Introducción.....	8
4.2. Relación con ISO/IEC 27034 y OWASP	8
4.3. Fases del S-SDLC.....	9
4.3.1. Fase 1: Requisitos.....	9
4.3.2. Fase 2: Diseño	9
4.3.3. Implementación/Coding	10
4.3.4. Fase 4: Testing/Verificación.....	10
5. OWASP Top 10 (Edición 2025).....	12
5.1. Contexto: qué es OWASP y propósito del Top 10.....	12
5.2. Metodología y novedades de la edición 2025	12
3.2.1. Cambios respecto a la edición 2021	12
5.3. Las diez categorías del Top 10:2025	13
5.3.1. A01:2025 – Broken Access Control.....	13
5.3.2. A02:2025 – Security Misconfiguration	13
5.3.3. A03:2025 – Software Supply Chain Failures	13
5.3.4. A04:2025 – Cryptographic Failures	13
5.3.5. A05:2025 – Injection	14

5.3.6. A06:2025 – Insecure Design	14
5.3.7. A07:2025 – Authentication Failures.....	14
5.3.8. A08:2025 – Software or Data Integrity Failures	14
5.3.9. A09:2025 – Security Logging & Alerting Failures	14
5.3.10. A10:2025 – Mishandling of Exceptional Conditions	14
5.4. Análisis técnico de las vulnerabilidades más críticas	15
5.4.1. Inyecciones (A05:2025)	15
5.4.2. Fallos de Autenticación (A07:2025).....	15
5.4.3. Exposición de Datos y Fallos Criptográficos (A04:2025).....	16
6. Seguridad en el despliegue	16
6.1. SAST – Static Application Security	16
6.1.1. Qué es SAST	16
6.1.2. Cómo funciona SAST.....	17
6.1.3. Ejemplos importantes de herramientas SAST	17
6.2. DAST – Dynamic Application Security Testing.....	17
6.2.1. Qué es DAST	17
6.2.2. Cómo funciona DAST	17
6.2.3. Ejemplos importantes de herramientas DAST	17
• OWASP ZAP. Open-source, muy integrable en CI/CD.....	17
• Burp Suite. El estándar de facto en pen testing manual/semi-automático.....	17
• Nikto. Scanner web ligero, ideal para chequeos.....	17
6.3. Ventajas y desventajas de SAST y DAST	17
6.4. Complementariedad entre SAST, DAST, y otras.....	18
6.5. Conclusión sobre DevSecOps	18
Bibliografía.....	19

1. Introducción

La seguridad del software se ha convertido en uno de los pilares fundamentales del desarrollo de aplicaciones modernas. En un contexto en el que los ciberataques son cada vez más frecuentes y sofisticados, no basta con construir sistemas funcionales: es imprescindible que sean seguros desde su concepción. La creciente digitalización de sectores críticos como la banca, la sanidad o la administración pública hace que cualquier vulnerabilidad en una aplicación pueda tener consecuencias graves, tanto para las organizaciones como para los usuarios finales.

Este trabajo aborda la seguridad de aplicaciones desde una perspectiva integral, combinando marcos normativos, metodologías de desarrollo y herramientas técnicas. En primer lugar, se analiza el estándar internacional ISO/IEC 27034, que proporciona un marco estructurado para integrar la seguridad a lo largo de todo el ciclo de vida del desarrollo software. A continuación, se estudia el Ciclo de Vida de Desarrollo Seguro (S-SDLC), que traslada esos principios normativos a la práctica, incorporando controles de seguridad en cada fase del proceso. Se examina también el OWASP Top 10 en su edición de 2025, referencia indispensable para conocer las vulnerabilidades más críticas a las que se enfrentan las aplicaciones web en la actualidad.

Finalmente, se profundiza en las técnicas y herramientas de análisis de seguridad empleadas en entornos DevSecOps, concretamente SAST y DAST, cuya integración en los pipelines de integración continua permite detectar y corregir vulnerabilidades de forma temprana y automatizada. El objetivo de este documento es ofrecer una visión coherente y estructurada de cómo abordar la seguridad del software no como una revisión final, sino como una responsabilidad compartida que atraviesa todo el proceso de desarrollo.

2. ISO/IEC 27034

El estándar ISO/IEC 27034 proporciona un estándar reconocido internacionalmente para la seguridad de las aplicaciones. Aunque todavía no se ha completado oficialmente, gran parte de la estructura del estándar ISO 27034 ya se establece a través de la publicación de la primera parte: ISO/IEC 27034-1: 2011.

La norma proporciona orientación para ayudar a las organizaciones a integrar la seguridad en los procesos utilizados para gestionar sus aplicaciones y es aplicable a aplicaciones desarrolladas internamente, aplicaciones adquiridas de terceros y donde el desarrollo o la operación de la aplicación se subcontrata.

La ISO 27034 está estrechamente alineada con varias otras normas ISO, particularmente ISO 27005 para la gestión de riesgos de seguridad de la información.

2.1. ¿Por qué es importante la ISO 27034?

La ISO 27034 es importante para la organización si se produce software y se desea estar conscientes de la seguridad de las mismas. Por ejemplo, las empresas que crean software

utilizado por servicios financieros, atención médica, sector público, defensa, aviación, dispositivos médicos y controles industriales, industrias de misión crítica probablemente se orientarán en la dirección de adherirse a la norma ISO 27034 con el tiempo.

Mientras tanto, el cumplimiento de la norma ISO 27034-1 es un diferenciador competitivo frente a la mayoría de las otras compañías de software. Esta es una de las razones por las cuales Microsoft, por ejemplo, anunció el cumplimiento de la norma ISO 27034-1 en 2013. Incluso si la ISO 27034 parece demasiado compleja para que adoptarla, el uso de sus conceptos como base para crear su programa de seguridad de aplicaciones probablemente ponga a la organización (y sus aplicaciones) por delante de los competidores.

2.2. Objetivo de ISO/IEC 27034

Esta norma multi-partes proporciona orientación sobre cómo especificar, diseñar/seleccionar y aplicar controles de seguridad de la información a través de un conjunto de procesos integrados a lo largo del Ciclo de Vida del Desarrollo de Software (SDLC) de una organización. Está orientada a los procesos.

ISO/IEC 27034-1 se basa en los siguientes principios fundamentales:

- La seguridad es un requisito
- La seguridad de las aplicaciones depende del contexto
- Inversión apropiada para aplicaciones de seguridad
- La seguridad en las aplicaciones debe ser demostrada

El objetivo es asegurar que aplicaciones informáticas ofrezcan el nivel deseado o necesario de seguridad y ofrece orientación sobre seguridad de la información en el diseño, desarrollo, programación e implementación de sistemas de aplicación.

Cubre aplicaciones de software desarrolladas internamente, mediante adquisición externa, tercerización / deslocalización o mediante enfoques híbridos. Aborda todos los aspectos, desde la determinación de los requisitos de seguridad de la información, hasta la protección de la información a la que accede una aplicación, así como la prevención del uso no autorizado y/o acciones de una aplicación.

El estándar es independiente del método SDLC: no exige uno o más métodos, enfoques o etapas de desarrollo específicos, pero está escrito de manera general para ser aplicable a todos ellos. De esta manera, complementa otros estándares y métodos de desarrollo de sistemas sin entrar en conflicto con ellos.

2.3. Cuadro normativo de la organización interesada en ISO/IEC 27034

- Definición del perímetro de la seguridad de una aplicación
- Relaciones y apoyo para implementar procesos de gestión de la seguridad de una aplicación
- Implementación de la norma ISO 27034 e integración en los procesos existentes

- Evaluación de los riesgos relacionados con la seguridad de una aplicación
- Realización, operación y validación de la aplicación de la seguridad durante todo el ciclo de vida
- Desarrollo de la validación de la seguridad de una aplicación
- Esbozo del proceso de certificación

2.4. Estructura del estándar: sus partes

La serie se compone de las siguientes partes:

- ISO/IEC 27034-1:2011 — Descripción general y conceptos
- ISO/IEC 27034-2:2015 — Marco normativo de la organización (ONF)
- ISO/IEC 27034-3:2018 — Proceso de gestión de seguridad de aplicaciones
- ISO/IEC 27034-5:2017 — Protocolos y estructura de datos de controles de seguridad de aplicaciones
- ISO/IEC TR 27034-5-1:2018 — Esquemas XML para los protocolos y estructura de datos
- ISO/IEC 27034-6:2016 — Estudios de casos

La parte 27034-6 proporciona ejemplos de cómo pueden desarrollarse y documentarse los controles de seguridad de aplicaciones (ASC) y define cómo debe manejarse la seguridad de la información en el curso del desarrollo de software.

Application Security Control (ASC): un ASC es un control que prescribe un conjunto de pasos a seguir al crear o actualizar una aplicación, previniendo vulnerabilidades de seguridad. Estos pasos garantizan que la información en el código fuente no sea modificada por nadie ajeno al creador de dicho código.

Relación entre componentes: el ONF y el ANF se utilizan para construir el ASMP. El ONF define las mejores prácticas de seguridad organizacional para todo el desarrollo de aplicaciones, mientras que el ANF las aplica a cada proyecto específico

3. El ONF (Organizational Normative Framework)

3.1 Definición

El Organizational Normative Framework (ONF) es la colección de documentos, políticas y recursos de seguridad de aplicaciones que refleja la estrategia de seguridad de la organización, de forma estructurada. Puede considerarse como un plano de seguridad personalizable que guía a desarrolladores, profesionales de seguridad y auditores en la construcción y gestión de aplicaciones seguras.

El ONF es un archivo a nivel organizacional de controles y procedimientos de seguridad de aplicaciones. Una biblioteca de ASC, que es un componente central del ONF, puede

ser utilizada por la organización para almacenar y actualizar los ASCs. El ONF también describe las circunstancias bajo las cuales un proyecto de desarrollo de aplicaciones debería ejecutar una actividad de seguridad determinada, como realizar una prueba de penetración.

3.2 Componentes del ONF

El ONF es el contenedor de todos los subcomponentes de las mejores prácticas de seguridad de aplicaciones, catalogadas y aprovechadas por una organización. El estándar define las siguientes categorías:

- Políticas y normas de seguridad: incluye todas las políticas, estándares y mejores prácticas de seguridad de aplicaciones adoptadas por la organización.
- Contexto normativo externo: incluye estándares, leyes y regulaciones que afectan a la seguridad de las aplicaciones.
- Tecnologías requeridas y disponibles: incluye tecnologías aplicables a la seguridad de las aplicaciones.
- Requisitos funcionales y soluciones: documenta los requisitos funcionales de TI de la organización y las soluciones apropiadas para abordarlos.

Según ISO/IEC 27034, un ONF debe contener también: criterios de clasificación de aplicaciones, listas de controles y líneas de base, modelos de amenazas, roles y responsabilidades, procedimientos de desarrollo y despliegue, y trazabilidad documental y de auditoría.

3.3 Relación ONF → ANF (Application Normative Framework)

Con el Application Normative Framework (ANF), las organizaciones crean una instancia específica del ONF para cada aplicación. Para ello, deben describir los requisitos y entornos/contextos específicos y evaluar los riesgos. Durante una auditoría, los auditores utilizan los ASCs definidos en el ANF para auditar tanto el proyecto como la aplicación.

El ASMP ayuda al equipo de proyecto a aplicar las partes relevantes del ONF a un proyecto de aplicación específico y a registrar formalmente la evidencia de los resultados en un ANF. A su vez, el ASMP apoya la mejora continua del ONF mediante la retroalimentación de nuevos conocimientos, sugerencias de mejora de controles y prácticas adquiridas en el curso de los proyectos.

3.4 Por qué el ONF es central en la seguridad de aplicaciones

El ONF facilita una seguridad contextualizada, en lugar de listas de verificación genéricas o herramientas de cumplimiento diseñadas para todos sin consideración del contexto propio. Cada organización es diferente: una aplicación de salud que maneja datos confidenciales de pacientes tendrá requisitos de seguridad completamente distintos a los de un sitio de catálogo de comercio electrónico.

El ONF permite a las organizaciones: centralizar y personalizar sus prácticas de seguridad para cumplir objetivos específicos y requisitos regulatorios. Esta personalización

garantiza que las medidas de seguridad sean escalables, adaptables y alineadas con las necesidades del negocio.

3.5 El ONF en el ciclo de vida de la aplicación

El ONF asegura la seguridad en cada etapa del SDLC:

Etapa	Contribución del ONF
Requisitos	Establece objetivos de seguridad según la clasificación de la app.
Diseño	Proporciona modelos de amenazas y guías arquitectónicas
Desarrollo	Define estándares de codificación segura y ASCs
Pruebas	Establece herramientas y técnicas de análisis de vulnerabilidades

4. Ciclo de Vida de Desarrollo Seguro (S-SDLC)

4.1. Introducción

El S-SDLC, o ciclo de vida de desarrollo seguro de software, no implica crear un proceso paralelo o independiente al ciclo de vida tradicional del desarrollo. En realidad, consiste en utilizar el mismo SDLC habitual, pero añadiendo actividades y controles de seguridad en cada una de sus fases: Requisitos, Diseño, Implementación y Verificación.

La idea principal es que la seguridad no debe tratarse como una revisión final antes de entregar el producto, sino como una parte integrada del desarrollo desde el inicio. Por ejemplo, en la fase de requisitos se pueden identificar necesidades de seguridad como autenticación, control de accesos o protección de datos. En la fase de diseño se pueden analizar posibles amenazas y definir una arquitectura más segura. Durante la implementación se deben seguir buenas prácticas de programación segura, evitando vulnerabilidades comunes como inyecciones SQL, errores de validación o gestión incorrecta de sesiones. Finalmente, en la verificación se realizan pruebas de seguridad, análisis de código y revisión de posibles fallos.

4.2. Relación con ISO/IEC 27034 y OWASP

La ISO/IEC 27034 marca que la seguridad de aplicaciones debe gestionarse de forma organizada y repetible, no como algo improvisado o que dependa únicamente del criterio de cada desarrollador.

El S-SDLC es la manera de aplicar eso en el ciclo real de desarrollo: desde que se definen requisitos hasta que se prueba la aplicación. Es decir, permite integrar la seguridad desde las primeras fases del proyecto, empezando por la definición de requisitos, continuando con el diseño de la arquitectura, la implementación del código y finalizando con la verificación mediante pruebas y revisiones de seguridad.

OWASP complementa esta idea proporcionando guías, estándares y herramientas prácticas. Por ejemplo, OWASP SAMM permite evaluar y mejorar la madurez de una organización en seguridad del software, adaptando la estrategia a sus riesgos y necesidades. Además, la OWASP Developer Guide estructura las buenas prácticas de desarrollo seguro en las fases dichas anteriormente.

4.3. Fases del S-SDLC

4.3.1. Fase 1: Requisitos

En la fase de requisitos, la seguridad debe definirse antes de empezar a programar, igual que se definen las funcionalidades principales de la aplicación. No basta con decir “la aplicación debe ser segura”, porque eso es demasiado general y no permite comprobar si realmente se ha cumplido. Lo importante es transformar la seguridad en requisitos claros, concretos y verificables.

Hay tres tipos de requisitos:

- **Requisito funcional:** Describe una acción que el sistema debe realizar, por ejemplo: “el usuario podrá iniciar sesión con correo y contraseña”. Sin embargo, desde el punto de vista de seguridad, ese requisito debe completarse con condiciones más específicas: cómo se validan las credenciales, cómo se almacenan las contraseñas.
- **Requisito no funcional:** Indican características de calidad del sistema, como rendimiento, disponibilidad, escalabilidad o trazabilidad. Dentro de ellos se suelen incluir muchos requisitos de seguridad, porque la seguridad afecta a cómo debe comportarse el sistema, no solo a qué funciones ofrece.
- **Requisitos de seguridad:** Concretan las medidas que debe cumplir la aplicación para proteger la información, los usuarios y el propio sistema. Estos requisitos pueden estar relacionados con autenticación, autorización, cifrado, gestión de sesiones, registro de eventos, validación de entradas o protección frente a ataques comunes.

En resumen, en la fase de requisitos el objetivo es anticiparse a los problemas. La seguridad deja de ser algo genérico o que se revisa al final, y pasa a formar parte del contrato inicial del software: qué debe protegerse, cómo debe protegerse y cómo se va a comprobar que realmente se ha protegido.

4.3.2. Fase 2: Diseño

En esta fase se definen aspectos importantes como la arquitectura general, los módulos de la aplicación, la base de datos, las APIs, los flujos de información, los roles de usuario, los permisos y las integraciones con servicios externos. Todas estas decisiones tienen impacto directo en la seguridad. Por ejemplo, no es lo mismo validar permisos solo en el frontend que hacerlo también en el servidor, la segunda opción es mucho más segura porque el cliente puede ser manipulado fácilmente.

La pregunta principal en esta fase sería: “¿Cómo podría fallar este diseño o cómo podría atacarlo un usuario malintencionado?”. Esto obliga al equipo a pensar como un atacante antes de escribir demasiado código.

Por ejemplo, en una aplicación de facturas, no basta con diseñar una pantalla donde el usuario vea sus documentos. También hay que pensar cómo se autentica ese usuario, cómo se comprueba que solo puede acceder a sus propias facturas, etc.

Aquí entra el modelado de amenazas, que consiste en analizar posibles riesgos antes de programar. Básicamente, se intenta responder a preguntas como: qué activos queremos proteger, quién podría atacar el sistema, por dónde podría hacerlo, qué impacto tendría el ataque y qué controles habría que aplicar para reducir ese riesgo.

En resumen, la fase de diseño no consiste únicamente en “dibujar” cómo será la aplicación. También implica revisar si esa estructura es segura, identificar posibles ataques y decidir controles antes de que el código dependa de una mala decisión arquitectónica. Una buena fase de diseño permite evitar vulnerabilidades difíciles de corregir más adelante, como problemas de autorización, exposición de datos sensibles o dependencias inseguras entre componentes.

4.3.3. Implementación/Coding

En la fase de implementación es cuando se escribe el código de la aplicación. Aquí la idea principal es que la seguridad no se deje para una revisión final, sino que se aplique mientras se programa. Es decir, cada desarrollador debe intentar escribir código seguro desde el primer momento, siguiendo buenas prácticas y usando controles que reduzcan el riesgo de vulnerabilidades.

También es importante mantener una cierta coherencia entre los desarrolladores. Si cada persona programa la autenticación, la autorización o la validación de datos de una forma distinta, pueden aparecer inconsistencias y vulnerabilidades. Por eso conviene usar componentes comunes, funciones reutilizables y criterios compartidos dentro del equipo.

Otro punto importante es no confiar en el cliente. Aunque el frontend limite ciertas acciones o valide formularios, el backend siempre debe volver a comprobar los datos y permisos, porque cualquier petición puede ser modificada antes de llegar al servidor.

En resumen, esta fase consiste en traducir los requisitos y el diseño seguro a código. La seguridad se aplica en las decisiones pequeñas del día a día: cómo se valida una entrada, cómo se consulta la base de datos, cómo se comprueba un permiso o cómo se gestiona un error. Si estas decisiones se toman bien desde el principio, se reducen muchos problemas antes de llegar a la fase de pruebas.

4.3.4. Fase 4: Testing/Verificación

En la fase de testing o verificación se comprueba que todo lo que se ha definido antes realmente se cumple. Es decir, no basta con haber escrito requisitos de seguridad, haber diseñado controles o haberlos implementado en el código; ahora hay que demostrar que funcionan correctamente.

Es importante destacar que probar la seguridad no significa únicamente ejecutar un escáner automático. Los escáneres son útiles porque permiten detectar muchos fallos

comunes, pero no sustituyen el análisis humano. Hay vulnerabilidades que dependen de la lógica de negocio o del diseño de la aplicación y que una herramienta automática puede no entender.

Tipos de pruebas de seguridad:

- Revisión de requisitos de seguridad: Primero se comprueba que los requisitos de seguridad definidos al inicio siguen siendo válidos y se han implementado. La pregunta sería: “¿La aplicación cumple lo que dijimos que debía cumplir?”
- Revisión de código: La revisión de código consiste en analizar cómo se ha programado la aplicación para detectar patrones inseguros. Aquí se pueden encontrar problemas como consultas SQL construidas concatenando texto, falta de validación de entradas, secretos guardados en el repositorio, errores que muestran información interna o comprobaciones de autorización mal implementadas.
- SAST - Static Application Security Testing: El SAST analiza el código fuente. Este tipo de análisis ayuda a encontrar vulnerabilidades antes de desplegar la aplicación. Su ventaja es que puede integrarse pronto en el desarrollo, incluso en pipelines de integración continua, para avisar al equipo antes de que el fallo llegue a producción.
- DAST – Dynamic Application Security Testing: El DAST analiza la aplicación mientras está en ejecución. En lugar de mirar el código, envía peticiones a la aplicación como si fuera un usuario externo o un atacante. Sirve para encontrar problemas que aparecen cuando la aplicación ya está funcionando en un entorno de pruebas.
- Pruebas manuales y pentesting: Las pruebas manuales siguen siendo necesarias porque las herramientas automáticas no entienden siempre la lógica de negocio de una aplicación. El pentesting va un paso más allá, porque intenta evaluar la aplicación desde la perspectiva de un atacante. Su objetivo no es solo encontrar errores técnicos, sino comprobar hasta dónde podría llegar alguien si explota una vulnerabilidad.

Estos tipos de pruebas no constituyen necesariamente una secuencia rígida, sino un conjunto de técnicas complementarias. Aun así, pueden organizarse de forma lógica desde la revisión de requisitos y código hasta el análisis automatizado y las pruebas manuales más avanzadas

En resumen, la fase de verificación sirve para confirmar que la seguridad no se ha quedado solo en teoría. Esta fase combina herramientas automáticas con revisión humana, porque ambas son necesarias para obtener una visión más completa de la seguridad.

5. OWASP Top 10 (Edición 2025)

5.1. Contexto: qué es OWASP y propósito del Top 10

OWASP (*Open Worldwide Application Security Project*) es una fundación sin ánimo de lucro dedicada a la seguridad del software. Su proyecto más reconocido, el **OWASP Top 10**, es un documento de concienciación que recoge las diez categorías de riesgo más críticas para las aplicaciones web. Se actualiza cada tres o cuatro años y es el estándar de facto que utilizan auditores y normativas internacionales (PCI-DSS 4.0, NIST 800-53/SSDF, SOC 2 e ISO 27001) como referencia mínima de cumplimiento.

El Top 10 no es un estándar completo de seguridad, sino una guía de concienciación. Para implantaciones rigurosas, OWASP recomienda combinarlo con marcos más amplios como SAMM, ASVS o DSOMM. Su valor reside en proporcionar un lenguaje común entre desarrollo, AppSec y dirección sobre dónde está realmente el riesgo.

5.2. Metodología y novedades de la edición 2025

La edición 2025 (octava del proyecto) se presentó en la *Global AppSec Conference* de Washington DC en noviembre de 2025. Se analizaron **más de 175.000 registros CVE** y datos de aproximadamente **2,8 millones de aplicaciones**, mapeando **248 CWEs** en diez categorías. La metodología combina datos cuantitativos (ocho categorías por incidencia y CVSS) con una encuesta a la comunidad (dos categorías por votación).

OWASP declara explícitamente que el ranking 2025 desplaza el foco *de los síntomas hacia las causas raíz*. Para el equipo de desarrollo, este matiz es importante: las contramedidas deben atacar la raíz arquitectónica, no parchear el síntoma observable.

3.2.1. Cambios respecto a la edición 2021

Posición 2021	Categoría 2021	Posición 2025	Categoría 2025	Movimiento
A01	Broken Access Control	A01	Broken Access Control	Absorve A10:2021
A02	Cryptographic Failures	A04	Cryptographic Failures	↓2
A03	Injection	A05	Injection	↓2
A04	Insecure Design	A06	Insecure Design	↓2
A05	Security Misconfiguration	A02	Security Misconfiguration	↑ 3
A06	Vulnerable and Outdated Components	A03	Software Supply Chain Failures	↑ 3 y se expande
A07	Identification and Authentication Failures	A07	Authentication Failures	Solo renombrada
A08	Software and Data Integrity Failures	A08	Software or Data Integrity Failures	Igual

A09	Security Logging and Monitoring Failures	A09	Security Logging & Alerting Failures	Solo renombrada
A10	Server-Side Request Forgery (SSRF)	-	-	Consolidada en A01
-	-	A10	Mishandling of Exceptional Conditions	Nueva

Las dos novedades de mayor calado son **A03 – Software Supply Chain Failures** (evolución de la antigua «Vulnerable and Outdated Components», amplía el alcance a todo el ciclo de construcción, distribución y actualización) y **A10 – Mishandling of Exceptional Conditions**, totalmente nueva, que cubre el manejo deficiente de errores, *failing open* y degradación insegura ante condiciones imprevistas.

5.3. Las diez categorías del Top 10:2025

5.3.1. A01:2025 – Broken Access Control

El control de acceso falla y un usuario actúa fuera de sus permisos: escalada de privilegios, IDOR, errores CORS, manipulación de tokens y, desde 2025, **SSRF** consolidado aquí. *Mitigación*: denegar por defecto, mínimo privilegio, centralizar la autorización en un único componente (gateway/middleware), validar la propiedad del recurso en cada operación, lista blanca de destinos y bloqueo de IPs internas para SSRF.

5.3.2. A02:2025 – Security Misconfiguration

Cuentas por defecto activas, servicios innecesarios, permisos abiertos, cabeceras de seguridad ausentes o mensajes de error verbosos. Sube del #5 al #2 por el auge del cloud y la IaC. *Mitigación*: procesos de *hardening* automatizados, imágenes mínimas, escaneo de IaC (Checkov, tfsec) en CI/CD, cabeceras Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options.

5.3.3. A03:2025 – Software Supply Chain Failures

Compromisos en construcción, distribución o actualización: dependencias maliciosas (*typosquatting*), mantenedores secuestrados, pipelines comprometidos. Ejemplos reales: event-stream (npm, 2018), xz-utils (2024), SolarWinds (2020). *Mitigación*: generar un **SBOM** (CycloneDX, Syft), firmar artefactos (Sigstore, Cosign), bloquear versiones con *lockfiles*, escaneo continuo (Dependabot, Snyk, OWASP Dependency-Check) y aislar el pipeline de CI.

5.3.4. A04:2025 – Cryptographic Failures

Ausencia o mal uso de la criptografía: contraseñas sin hashing adecuado, algoritmos obsoletos (MD5, SHA-1, DES), claves *hardcoded*, datos en tránsito sin TLS. Sustituye al antiguo «Sensitive Data Exposure» trasladando el foco a la causa raíz. *Mitigación*: hashing con **Argon2id** (alternativas: bcrypt, scrypt); cifrado AES-256-GCM y TLS 1.3 obligatorio; gestión de claves en KMS (Vault, AWS KMS) nunca en el repositorio; usar librerías auditadas (libsodium) y no implementar criptografía propia.

5.3.5. A05:2025 – Injection

El sistema incorpora datos no confiables a un intérprete (SQL, shell, LDAP, *templating*, NoSQL) sin sanitizar. Cae del #3 al #5 pero sigue siendo de las categorías con mayor número de CWEs (38) e impacto más severo. *Mitigación*: desarrollada en el apartado 3.4.1.

5.3.6. A06:2025 – Insecure Design

Fallos de diseño, no de implementación: flujos de recuperación débiles, ausencia de modelado de amenazas, falta de pasos de autorización, *rate-limiting* inexistente. No se corrige con un parche: requiere rediseñar. *Mitigación*: *threat modeling* en fase de diseño (STRIDE, PASTA), requisitos de seguridad explícitos en historias de usuario, revisiones de arquitectura previas a la implementación, uso del **ASVS** como *checklist* por nivel de criticidad.

5.3.7. A07:2025 – Authentication Failures

Verificación deficiente de la identidad: contraseñas débiles aceptadas, ausencia de MFA, gestión de sesión deficiente, susceptibilidad a *credential stuffing* y *brute force*. *Mitigación*: desarrollada en el apartado 3.4.2.

5.3.8. A08:2025 – Software or Data Integrity Failures

Plugins, actualizaciones o datos cargados sin verificar autenticidad e integridad; deserialización insegura. Se diferencia de A03 en que se centra en los **límites de confianza dentro del sistema**, no en la cadena externa. *Mitigación*: firmas digitales en artefactos, **Subresource Integrity (SRI)** en CDN, evitar deserialización de fuentes no confiables (preferir JSON con validación de esquema sobre Pickle/Java serialization), pipelines con *attestation* (SLSA, in-toto).

5.3.9. A09:2025 – Security Logging & Alerting Failures

Logs insuficientes, no monitorizados o manipulables, ausencia de alertas para actividad sospechosa. El nuevo nombre subraya que *logging sin alerting carece de valor*. *Mitigación*: logs estructurados (JSON) con correlación por *trace-id*, centralización en SIEM (ELK, Splunk, Wazuh), *alerting* con umbrales calibrados, almacenamiento *append-only* (WORM) y plan de respuesta a incidentes documentado.

5.3.10. A10:2025 – Mishandling of Exceptional Conditions

Gestión deficiente de situaciones excepcionales: *failing open*, fugas en *stack traces*, condiciones de carrera, agotamiento de recursos. Solo se manifiestan bajo estrés y pasan desapercibidos a la mayoría de escáneres. *Mitigación*: **fail closed** (denegar por defecto ante cualquier anomalía), mensajes de error genéricos al usuario con detalle solo en logs internos, *circuit breakers* y *timeouts* explícitos, pruebas específicas de paths de error con *chaos engineering* y *fault injection*.

5.4. Análisis técnico de las vulnerabilidades más críticas

Se profundizará en **Inyecciones, Fallos de Autenticación y Exposición de Datos**.

5.4.1. Inyecciones (A05:2025)

SQL Injection. Mitigación canónica: **consultas parametrizadas**. El motor recibe estructura y datos por canales separados, eliminando la posibilidad de que el dato altere la consulta.

```
python

# Vulnerable
query = f"SELECT * FROM users WHERE email='{email}'"
cursor.execute(query)

# Seguro: prepared statement
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Complementos: ORM con *bindings* (SQLAlchemy, Hibernate), validación de tipos y mínimo privilegio en la cuenta de BD (nunca conectar como root).

Cross-Site Scripting (XSS). Mitigación: codificar la salida según el contexto (HTML, atributo, JavaScript, URL).

```
javascript

element.innerHTML = userInput;      // Vulnerable
element.textContent = userInput;    // Seguro
```

Refuerzos: **Content Security Policy** estricta (script-src 'self' 'nonce-xxx'), frameworks modernos que escapan por defecto (React, Vue), y DOMPurify cuando se necesite HTML enriquecido.

Command Injection. Evitar invocar el shell; usar APIs nativas con argumentos como lista:

```
python

os.system(f"ping {host}")           # Vulnerable
subprocess.run(["ping", "-c", "1", host], shell=False) # Seguro
```

5.4.2. Fallos de Autenticación (A07:2025)

Las contramedidas se agrupan en cuatro frentes:

1. **Almacenamiento de credenciales:** hashing con **Argon2id** (parámetros OWASP 2024: 19 MiB, 2 iteraciones, paralelismo 1); alternativas válidas, bcrypt (coste 12+) o scrypt. Nunca SHA-256/MD5.
2. **Política de contraseñas (NIST SP 800-63B):** longitud mínima 12, sin reglas de complejidad obligatoria, verificación contra listas de contraseñas comprometidas (HaveIBeenPwned API).

3. **MFA:** preferentemente WebAuthn/FIDO2 (resistente a *phishing*); TOTP como alternativa; evitar SMS cuando sea posible.
4. **Gestión de sesión:** tokens generados con CSPRNG (≥ 128 bits); cookies con HttpOnly, Secure, SameSite=Lax; rotación de ID tras login (anti *session fixation*); expiraciones idle (15-30 min) y absoluta (8-24 h). Si se usa JWT: firmas RS256/EdDSA, exp corto y *refresh tokens*.

Además, defensa contra ataques automatizados: *rate-limiting* por IP y por cuenta, *account lockout* progresivo y CAPTCHA tras N intentos fallidos.

5.4.3. Exposición de Datos y Fallos Criptográficos (A04:2025)

«Exposición de Datos» fue el nombre histórico (Top 10 hasta 2017) de lo que ahora OWASP llama Cryptographic Failures. La mitigación se estructura en tres ejes:

- **Datos en reposo:** cifrado a nivel de almacenamiento (LUKS, BitLocker) y de aplicación; cifrado *envelope* con KMS (la clave maestra KEK protege las claves de datos DEK); tokenización para PCI-DSS.
- **Datos en tránsito:** TLS 1.3 exclusivamente, deshabilitar TLS 1.0/1.1 y SSL; HSTS con max-age ≥ 6 meses e includeSubDomains; *certificate pinning* en aplicaciones móviles críticas; *forward secrecy* con suites ECDHE.
- **Datos en uso:** minimización (alineación con RGPD, art. 5.1.c), pseudonimización, no registrar nunca datos sensibles en logs (PAN, contraseñas, tokens) aplicando *redaction* automática en el *logger*.

6. Seguridad en el despliegue

El desarrollo de software tradicional ponía el foco de la seguridad en el desarrollador. Son bien conocidos los cursos de seguridad en el desarrollo que se imparten para evitar problemas graves para las organizaciones que se dedican a construir software. Sin embargo, los errores humanos son inevitables, por lo que confiar únicamente en la formación de los desarrolladores no es suficiente. Por ello, nace la filosofía DevOps.

Esta sección se va a centrar en un subapartado de esta filosofía, que ayuda al cumplimiento del ciclo de vida de desarrollo seguro. Se trata de DevSecOps, que surge como alternativa a auditar el software al terminar, haciendo un “*shift left*” de estos controles de seguridad lo más a la izquierda posible temporalmente. Así, se encuentran vulnerabilidades más precozmente, reduciendo el coste de subsanarla considerablemente.

Para la implementación de DevSecOps, existen dos herramientas fundamentales, que cubren diferentes aspectos de la seguridad en distintos tiempos: SAST (*Static Application Security Testing*) y DAST (*Dynamic Application Security Testing*).

6.1. SAST – Static Application Security

6.1.1. Qué es SAST

Las **Pruebas de Seguridad de Aplicaciones Estáticas** son una técnica de ciberseguridad proactiva que analiza el código fuente antes de su ejecución. Como tienen acceso al código, son pruebas de **caja blanca**. Examina la estructura en busca de vulnerabilidades, como inyección SQL, XSS o credenciales codificadas.

Las herramientas SAST se integran comúnmente en los pipelines de **CI/CD** para automatizar el escaneo con cada cambio en el repositorio, proporcionando a los desarrolladores retroalimentación inmediata sobre prácticas inseguras.

6.1.2. Cómo funciona SAST

Es importante entender el funcionamiento de estas herramientas para comprender de forma más sencilla su alcance. Para analizar el código, sigue varios pasos:

1. Contruye un AST (Abstract Syntax Tree) o un grafo de flujo de datos.
2. Aplica reglas sobre patrones peligrosos, como llamadas a funciones inseguras, flujo de datos sin sanitizar, concatenación directa con consultas a la base de datos, etc.
3. Se aplican técnicas avanzadas, como **taint analysis** (rastrea desde dónde viene un dato hasta dónde llega).

6.1.3. Ejemplos importantes de herramientas SAST

- SonarQube. El más extendido en entornos empresariales. Es multilenguaje.
- Semgrep. Basado en reglas personalizables, muy popular en DevSecOps moderno.
- Checkmarx. Solución enterprise, muy completa.
- Snyk Code. Integra SAST con SCA.
- Bandit. SAST específico para Python.

6.2. DAST – Dynamic Application Security Testing

6.2.1. Qué es DAST

Las Pruebas de Seguridad Dinámicas constituyen otra técnica de seguridad, diferente a las pruebas estáticas. Se diferencian de estas últimas en que analizan la aplicación en tiempo de ejecución, sin acceso real al código, por lo que se comportan como pruebas de **caja negra**. Se le puede considerar como la forma de **pentesting** automática.

6.2.2. Cómo funciona DAST

Estas potentes herramientas ejercen diferentes acciones contra el sistema, con las siguientes características:

1. Actúa como un **atacante externo** — solo ve lo que ve la aplicación desde fuera (caja negra)
2. Envía payloads maliciosos: inyecciones SQL, XSS, path traversal, etc.
3. Analiza las respuestas: códigos de error, contenido de la respuesta, comportamientos anómalos
4. Puede actuar como **proxy** intercalado entre el cliente y el servidor para inspeccionar y modificar el tráfico

6.2.3. Ejemplos importantes de herramientas DAST

- OWASP ZAP. Open-source, muy integrable en CI/CD
- Burp Suite. El estándar de facto en pen testing manual/semi-automático
- Nikto. Scanner web ligero, ideal para chequeos

6.3. Ventajas y desventajas de SAST y DAST

Ambos tipos de herramientas cuentan con conjuntos de ventajas muy positivas, pero también con desventajas, entre las que se encuentran la falta de alcance de cada una. No obstante, veremos que el uso de ambas en conjunto puede suplir este factor negativo.

Las herramientas SAST son muy eficientes para **desvelar problemas pronto**, en el propio entorno de desarrollo o en la *Pull Request*. Además, cubre por completo el código, y se puede integrar fácilmente en los pipelines de CI. No obstante, puede detectar **falsos positivos**, no detecta vulnerabilidades de configuración o tiempo de ejecución, y tiene que estar adaptada al lenguaje de programación.

Por otra parte, las herramientas DAST **cubren las vulnerabilidades en tiempo de ejecución**, es agnóstico al lenguaje, y suele tener muy pocos falsos positivos, porque si consigue explotar alguna vulnerabilidad, es real. Sin embargo, tiene algunas desventajas con respecto a SAST, ya que no cubre todo el código, necesita un entorno desplegado y es más lento.

6.4. Complementariedad entre SAST, DAST, y otras

Ninguno de los dos es suficiente solo. La práctica estándar es usarlos en capas. Primero, se prueba el código fuente con SAST, y una vez se haya probado, se construye y despliega en desarrollo, donde se prueba la aplicación con DAST.

Existen herramientas llamadas IAST (*Interactive AST*), que combinan ambos enfoques instrumentando la aplicación en tiempo de ejecución para obtener el contexto interno mientras se ejecutan los tests.

Por último, y de una considerable importancia, también se emplean herramientas SCA (*Software Composition Analysis*), cuya función es analizar dependencias de terceros, un vector de ataque enorme.

6.5. Conclusión sobre DevSecOps

La clave de DevSecOps es que estos controles son **automáticos y bloqueantes** — un PR con una severidad alta en SAST no puede mergearse, igual que si falla un test unitario. La seguridad deja de ser un proceso separado y se convierte en parte del flujo de desarrollo. SAST y DAST ofrecen, en conjunto, la implementación técnica concreta para insertar la metodología S-SDLC en el pipeline.

Bibliografía

OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>

OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/2021/>

OWASP Foundation. (2025). *OWASP Application Security Verification Standard (ASVS) 5.0*.

Cyber World YT. *OWASP TOP 10 - EXPLAINED WITH EXAMPLES - 2024 [Video]*. (2024, 27 de marzo). YouTube. <https://www.youtube.com/watch?v=Gb7RDognNcc>

ISO/IEC 27034-1:2011/Amd 1:2024 — *Information technology — Application security — Part 1: Overview and concepts*.

NIST SP 800-63B — *Digital Identity Guidelines: Authentication and Lifecycle Management*.

MITRE Corporation. *Common Weakness Enumeration (CWE) — versión 4.18*. <https://cwe.mitre.org/>

OpenText. (s.f.). *¿Qué es SAST?* <https://www.opentext.com/es/what-is/sast>

Wiz. (s.f.). *Static Application Security Testing (SAST)*. <https://www.wiz.io/es-es/academy/application-security/static-application-security-testing-sast>

Trend Micro. (s.f.). *SAST: Static Application Security Testing*. https://www.trendmicro.com/es_es/what-is/proactive-security/sast-static-application-security-testing.html

OWASP Top 10 Vulnerabilities Explained - [OWASP Top 10 Vulnerabilities Explained | Stay Secure in 2025!](#)

OWASP Foundation. (2025). *OWASP Application Security Verification Standard (ASVS) 5.0*. https://owasp.org/www-project-application-security-verification-standard/?utm_source

ISO/IEC. (2011). *ISO/IEC 27034-1:2011 — Information technology — Security techniques — Application security — Part 1: Overview and concepts*. https://www.iso.org/standard/44378.html?utm_source